

Hacia la Continuidad de Modelos para Sistemas Multi-Robot: Integración de EB-DEVS con ROS2

Ezequiel Pecker-Marcosig
FI-UBA - ICC UBA/CONICET
Buenos Aires, Argentina
epecker@fi.uba.ar

J. Francisco Presenza
FI-UBA - INTECIN UBA/CONICET
Buenos Aires, Argentina
jpresenza@fi.uba.ar

Manuel Núñez Mosqueda
FI-UBA
Buenos Aires, Argentina
mnunezm@fi.uba.ar

Rodrigo Castro
FCEyN-UBA - ICC UBA/CONICET
Buenos Aires, Argentina
rcastro@dc.uba.ar

Abstract—Las metodologías de diseño basadas en modelado y simulación han adquirido una creciente relevancia en aplicaciones robóticas, al permitir la validación temprana de estrategias de control antes de su despliegue sobre hardware real. En el ecosistema ROS, las aplicaciones suelen estructurarse en nodos con funciones específicas –como control, percepción, navegación o fusión de datos– que son validados mediante simuladores como Gazebo o Isaac Sim. Sin embargo, estas herramientas presentan limitaciones para la simulación de Sistemas Multi-Robot Interconectados (NMRS), donde la coordinación depende de la interacción entre movilidad, sensado y comunicaciones. Este trabajo presenta las modificaciones realizadas en la herramienta de simulación EB-DEVS y la extensión de su biblioteca para NMRS para permitir su integración con ROS2. La propuesta habilita la ejecución de modelos de control desarrollados en EB-DEVS dentro de arquitecturas ROS, preservando los principios de continuidad de modelos entre las etapas de simulación y despliegue. La plataforma resultante fue validada experimentalmente mediante la implementación de estrategias de seguimiento de waypoints sobre un robot unicycle con dirección diferencial, demostrando la viabilidad de utilizar EB-DEVS como entorno de diseño y verificación de controladores para aplicaciones integradas con ROS2.

I. INTRODUCCIÓN Y TRABAJOS RELACIONADOS

El diseño de controladores para sistemas ciberfísicos y sistemas embebidos que interactúan con su entorno, suele ser una tarea desafiante. Validar estrategias de control directamente sobre el hardware objetivo resulta arriesgado, ya que un diseño incorrecto puede provocar fallas graves y ocasionar daños irreparables. Además, si durante la etapa de diseño todavía no se dispone de todo el hardware necesario, entonces no se pueden hacer todas las pruebas a tiempo, lo que provoca retrasos en el ciclo de desarrollo del producto o sistema.

Por estos motivos, las técnicas de diseño basadas en modelado y simulación (M&S) han ganado terreno, dado que permiten adelantar la validación de las leyes de control a etapas tempranas de diseño [1]. Metodologías como *Model-In-The-Loop* (MIL), *Software-In-The-Loop* (SIL) o *Hardware-In-The-Loop* (HIL) comparten una etapa inicial de diseño en la cual se formula una ley de control analítica; a continuación, se construye un modelo en simulación del controlador, y

finalmente se verifica su desempeño aplicando este último sobre un modelo en simulación de la planta a controlar. Sin embargo, una vez validado, este modelo del controlador utilizado durante las etapas de diseño suele descartarse y reimplementarse en el hardware final, introduciendo una brecha entre simulación e implementación. Esta discontinuidad puede provocar inconsistencias y errores de implementación. Para evitarla, en [2] se propuso la *metodología de continuidad de modelos*, donde el mismo modelo de simulación del controlador se reutiliza a lo largo de todas las fases del proceso de diseño [3]. Esto incluye la ejecución de una simulación embebida en el hardware objetivo que involucra al simulador y al modelo del controlador. En este caso, el simulador debe tener la capacidad de ejecutarse en un sistema embebido con recursos de cómputo limitados, en tiempo real, y ser capaz de proveer las abstracciones necesarias en las interfaces de entrada/salida del modelo de control, de modo que éste interactúe de forma transparente con un modelo de la planta o con la planta real.

Esta idea no es extraña en robótica, donde ROS (Robot Operating System) [4] proporciona una abstracción basada en nodos y mensajes que facilita una transición gradual desde la simulación al robot real. Típicamente, el controlador se desarrolla como nodos ROS, se valida en simuladores como Gazebo o Isaac Sim, y luego se despliega sobre el hardware real reutilizando las mismas interfaces. Aunque Gazebo e Isaac Sim ofrecen modelos precisos de la dinámica y el sensado de los robots, carecen de mecanismos realistas para representar ciertos comportamientos asociados a las interacciones entre múltiples robots. Un ejemplo claro de esta limitación es la modelización de redes de comunicaciones dinámicas, cuyas topologías varían continuamente debido al movimiento de los robots y a las restricciones de conectividad, como el rango de comunicación. Esta limitación resulta crítica en sistemas multi-robot cooperativos, donde la coordinación depende fuertemente de la evolución de la red de comunicaciones y del acoplamiento entre movilidad y conectividad.

Para abordar esta problemática, en la literatura se han propuesto diversas plataformas de co-simulación con simuladores

de comunicaciones, como DANCERS [5], CORNET [6], y ChoiRBot [7], entre otros. Sin embargo, estas herramientas todavía presentan numerosos desafíos. Por ejemplo, los mecanismos necesarios para sincronizar los relojes y los estados de los robots en los distintos simuladores involucrados introducen complejidad y pueden conducir a una pérdida de rendimiento.

Un enfoque alternativo consiste en desarrollar una herramienta de simulación unificada, capaz de representar en un mismo marco tanto la dinámica de movimiento y sensado de los robots como la de los enlaces de comunicación [8]. Recientemente [9] presentó un entorno unificado para la simulación y validación de controladores, específicamente orientado a Sistemas Multirobot Interconectados (Networked Multirobot Systems, NMRS). El mismo se fundamenta en el formalismo matemático *Emergent Behavior-DEVS* (EB-DEVS) [10], lo que permite integrar y equilibrar las ventajas de los enfoques existentes, al combinar facilidad de uso con la capacidad para modelar comportamientos complejos emergentes. Este entorno amplía las ideas propuestas en [7] para modelar redes de comunicación dinámicas, mediante la creación de un módulo que representa el medio de transmisión. Dicho módulo se encarga de enrutar los mensajes de cada robot hacia sus vecinos en función de su estado físico—posiciones, orientaciones, etc.—y de las especificaciones de sus sensores—alcance, campo visual, etc. Esto permite al sistema imitar el comportamiento real de los robots que transmiten mensajes sin tener un conocimiento explícito de sus vecinos. Sin embargo, la principal limitación de esta herramienta es que carece de conexión con ROS.

El principal aporte de este trabajo es la extensión del entorno de simulación EB-DEVS para NMRS [9] para su integración con ROS2. El objetivo es habilitar el diseño de controladores para NMRS siguiendo la metodología de *continuidad de modelos*. Esta propuesta se validó experimentalmente en el desarrollo del control de seguimiento de waypoints aplicado a un robot unicycle con dirección diferencial.

Este trabajo se estructura de la siguiente manera. La Sección II introduce los conceptos necesarios para una mejor comprensión del trabajo. En la Sección III-A se presentan las modificaciones introducidas en EB-DEVS para integrarse con ROS2 y la extensión de la biblioteca de modelos presentada en [9], mientras que la Sección III-B introduce la arquitectura del control. La Sección IV presenta los resultados experimentales de aplicar la herramienta extendida para el diseño de un control con una metodología de continuidad de modelos. Finalmente, la Sección V presenta las conclusiones.

II. CONCEPTOS PRELIMINARES

A. Formalismo Emergent Behavior-DEVS (EB-DEVS)

La Especificación de Sistemas de Eventos Discretos (DEVS) [11] es el formalismo matemático más general para la representación de sistemas de eventos discretos (que reciben secuencias de eventos de entrada y producen secuencias de eventos de salida). DEVS permite aproximar sistemas continuos con cualquier grado de exactitud deseado mediante

los métodos numéricos QSS [12]. Además, permite la representación de sistemas dinámicos de naturaleza híbrida bajo un único formalismo matemático en un modelo unificado.

Un sistema modelado con DEVS se describe como una composición modular y jerárquica de submodelos de comportamiento (*atómicos*) y de estructura (*acoplados*). El comportamiento de un modelo atómico DEVS se define mediante tres conjuntos: (a) uno para los *eventos de entrada*, (b) uno para los *eventos de salida* y (c) uno para los *estados internos*, y cuatro funciones dinámicas: (a) la *función de transición interna* para el comportamiento autónomo, (b) la *función de transición externa* para el comportamiento reactivo, (c) la *función de salida* para la emisión de eventos de salida, y (d) la *función de avance de tiempo* para la duración de los estados. Un modelo acoplado DEVS define una red de modelos que interactúan, organizados en una estructura modular y jerárquica.

El formalismo EB-DEVS [10] es una extensión de DEVS para lidiar con propiedades emergentes en sistemas multiagente. Para lograrlo, EB-DEVS incorpora un estado global en los modelos acoplados y lazos de realimentación multinivel, permitiendo que los modelos acoplados (padres) intercambien valores que son funciones de sus estados globales con sus modelos hijos (y viceversa). Esto permite un mecanismo de comunicación indirecto, con canales para la causación ascendente y la información descendente, a la vez que preserva el principio de ocultamiento de los estados internos de los modelos DEVS.

B. Biblioteca de Modelos EB-DEVS para NMRS

El simulador de modelos EB-DEVS también se llama EB-DEVS, y los modelos se describen en lenguaje Python. Recientemente fue presentada una biblioteca extensible de modelos EB-DEVS destinada a dar soporte a aplicaciones relacionadas con NMRS y estrategias de control distribuido [9]. Estos modelos fueron diseñados teniendo en cuenta la modularidad, lo que permite combinarlos para construir una amplia gama de aplicaciones complejas, aprovechando la estructura modular inherente a DEVS.

La arquitectura completa del sistema multirobot está contenida en el modelo acoplado de nivel superior llamado `MultiRobotSystem`. Este modelo consta de N_r instancias del modelo acoplado `Robot` con enlaces bidireccionales al modelo atómico `TransmissionMedium`, el cual representa el entorno físico a través del cual se produce la comunicación entre los robots. El estado global del acoplado `MultiRobotSystem` es un diccionario indexado por robot que se actualiza con la información recibida de forma asincrónica de cada robot, incluyendo sus posiciones junto con una etiqueta de tiempo y el radio de alcance de la comunicación.

Esta biblioteca adopta un enfoque de modelado basado en agentes (ABM), en el que cada robot (agente) es un modelo acoplado que ejecuta un conjunto de reglas internas e interactúa con los agentes vecinos para intercambiar información. El modelo de los agentes incluye la física de los robots, los algoritmos de control, los

módulos de comunicación que intercambian datos, los sensores y los módulos de estimación del estado del robot. Por lo tanto, el modelo acoplado `Robot` incluye modelos atómicos para los subsistemas que se utilizan habitualmente en aplicaciones con múltiples robots, incluyendo la dinámica del robot acoplada (`Robot Dynamics`) y elementos atómicos como Localización (`Localization`), Control (`Control`), Coordinación (`Coordination`), Comunicación (`Communication`), y tanto Sensores Interoceptivos (`Interoceptive Sensor`) como Exteroceptivos (`Exteroceptive Sensor`). El acoplado `Robot` cuenta con un puerto de entrada `msg_in` y un puerto de salida `msg_out` que se utilizan para la comunicación con otros robots. El modelo acoplado `RobotDynamics` se utiliza para resolver numéricamente la ecuación diferencial ordinaria que describe el modelo dinámico del robot. Este modelo incluye un modelo atómico `DynamicsFunction` y tantos atómicos integradores (`mQSSI`, método numérico `QSS1` [12]) como estados tenga el modelo.

El modelo atómico del `TransmissionMedium` está diseñado para permitir una comunicación entre robots considerando la topología dinámica de la red. Para esto, el modelo acoplado de cada robot se conecta a `TransmissionMedium` mediante enlaces bidireccionales. Esto permite evitar establecer conexiones directas entre robots, lo que reduce el número total de conexiones y de eventos de $O(N_r^2)$ a $O(N_r)$. Cuando este atómico recibe un evento de entrada que transporta un *token* procedente de un robot *i*, este se enruta a los robots vecinos. Para obtener el conjunto de vecinos en el momento en que recibe el token, el medio de transmisión consulta el estado global del sistema multi-robot en donde se almacena el estado de cada robot.

C. ROS2 y MicroROS

ROS (*Robot Operating System*) es un marco de desarrollo ampliamente utilizado en robótica que facilita la construcción de sistemas modulares a partir de procesos independientes llamados *nodos*. Cada nodo implementa una función específica y se comunica con otros nodos mediante mensajes, utilizando patrones de comunicación. Esta organización favorece la reutilización de componentes, la separación funcional y la depuración incremental del sistema. ROS 2 mantiene este paradigma, pero adopta una arquitectura completamente distribuida basada en DDS, permitiendo la comunicación entre nodos que se ejecutan en una o múltiples plataformas conectadas en red, sin depender de un nodo maestro central [4].

ROS2 provee tres mecanismos para comunicar nodos: *tópicos*, *servicios* y *acciones*. Los *tópicos* siguen un esquema publicador/suscriptor, común en sistemas distribuidos, que permite una comunicación asíncrona unidireccional de uno a muchos. Cada vez que se publica un mensaje en un tópico, se ejecutan las funciones de *callback* de los nodos suscriptos al mismo. Los *servicios* son adecuados para la comunicación sincrónica tipo cliente/servidor entre nodos. Los servicios también tienen asociadas funciones de *callback*. Tanto los servicios como los tópicos utilizan mensajes de ROS que son

estructuras de datos fuertemente tipadas. Para este trabajo adoptamos la distribución Humble de ROS2 debido a su condición de versión con soporte de tiempo extendido (LTS).

MicroROS [13] (uROS) es una extensión de ROS2 para su utilización directamente en microcontroladores y sistemas embebidos con recursos limitados, permitiendo que se comuniquen de forma nativa dentro del ecosistema ROS2 mediante publicaciones y suscripciones. Las plataformas soportadas actualmente incluyen las placas de desarrollo de las familias Arduino, Teensy y ESP32. En este último caso, las interfaces de red de la ESP32 permiten correr un nodo completo de ROS embebido y comunicarse por WiFi con el resto de los nodos de forma distribuida. Para trabajar con uROS sobre FreeRTOS se debe definir una tarea periódica de *spin*, encargada de procesar las activaciones de los *timers*, las suscripciones a tópicos y la ejecución de las funciones de *callback* asociadas. Para llevar a cabo la integración de uROS en el entorno de desarrollo de la ESP32, es necesario incorporar el *componente* oficial de `micro_ros`. Es fundamental que la distribución elegida sea compatible con la versión del *framework* ESP-IDF utilizada.

III. INTEGRACIÓN EB-DEVS / ROS2

En esta sección se abordan las adaptaciones realizadas en el simulador EB-DEVS para integrar esta herramienta de simulación con ROS2.

A. Adaptaciones Realizadas al Simulador EB-DEVS y extensión de la biblioteca para NMRS

Para establecer un mecanismo de comunicación entre ROS2 y el simulador EB-DEVS, se modificó este último para que al lanzar la simulación se cree un *hilo secundario* para la ejecución de EB-DEVS cediendo el *hilo principal* para el ciclo principal (*spin*) de ROS2, y se extendió la biblioteca de modelos descrita en la Sección II-A con dos modelos atómicos: `Publisher` y `Subscriber`. Estos atómicos están basados en el desarrollo de [14] para el simulador PythonPDEVS.

En primer lugar, el atómico `Publisher` tiene un puerto de entrada `inp`, por donde recibe eventos generados dentro del simulador, y ningún puerto de salida. Recibe como parámetros un objeto `ROS2Client` que es un nodo de ROS2 y el tópico en donde publicar. Cada vez que recibe un evento en `inp`, ejecuta su función de transición externa para publicar el mensaje en el tópico correspondiente (con el método `publish` del objeto `ROS2Client` que recibió como parámetro).

En segundo lugar, para poder generar y enviar eventos desde ROS2 a los puertos de entrada de un modelo atómico EB-DEVS, es necesario ejecutar la simulación en tiempo real y exponer sus puertos de entrada. Para ello, el *hilo principal* se encarga de monitorear la llegada de mensajes de ROS2 publicados en los tópicos de interés y mapearlos en eventos en los puertos de los modelos EB-DEVS correspondientes cuya simulación se ejecuta en un *hilo secundario*. Para este fin, se desarrolló el atómico `Subscriber`, que tiene un puerto de entrada `inp` y un puerto de salida `out`. El puerto `inp` se expone para poder ser accedido desde fuera del simulador, en el hilo principal, y así inyectar eventos generados desde

ROS2. Cada vez que se publica un mensaje en uno de los tópicos de interés, se ejecuta la función de *callback* asociada y desde ella se introducen los mensajes como eventos en el puerto de entrada expuesto. Esto produce la ejecución de la función de transición externa del atómico *Subscriber*, que agenda instantáneamente un evento de salida en el puerto *out* para comunicar el mensaje recibido en ROS2 al resto de los modelos dentro de la simulación.

B. Arquitectura del Controlador

La Fig. 1 ilustra la arquitectura de control propuesta para un robot, estructurada en dos niveles. En el *nivel superior* reside el control cinemático¹ que, a partir de la pose estimada del robot $\hat{x} = (\hat{x}, \hat{y}, \hat{z}, \hat{\theta}_x, \hat{\theta}_y, \hat{\theta}_z) \in \mathbb{R}^6$ y de la trayectoria nominal que debe seguir el robot genera los comandos de velocidad lineal $v^{\text{cmd}} \in \mathbb{R}^3$ y de velocidad angular $\omega^{\text{cmd}} \in \mathbb{R}^3$ que debe seguir el robot. En este trabajo, la trayectoria se parametriza mediante una secuencia de *waypoints* $w[k] = (w_x[k], w_y[k], w_z[k], w_{\theta_x}[k], w_{\theta_y}[k], w_{\theta_z}[k]) \in \mathbb{R}^6$, $k = 1, \dots, N_w$. Este nivel se ejecuta en una computadora de placa única Raspberry Pi 4. Se eligió esta plataforma porque tiene la capacidad de ejecutar simulaciones con la herramienta EB-DEVS en tiempo real.

Por otro lado, el *nivel inferior* cuenta con un filtro de Kalman que fusiona los datos provenientes de distintos sensores (por ejemplo, *encoders*, IMU, y magnetómetro) para estimar $\hat{x}$. Este nivel cuenta además con el controlador encargado de accionar los actuadores del robot para hacer seguimiento de las velocidades $v^{\text{cmd}}, \omega^{\text{cmd}}$. Este nivel está implementado en una placa de desarrollo ESP32-Wroom-32 programados en C, dedicada exclusivamente a esta tarea.

La comunicación entre los controladores de alto nivel (Raspberry Pi 4) y bajo nivel (ESP32) se realiza de forma inalámbrica (WiFi) mediante ROS2 y uROS. Del lado de la Raspberry Pi, existen dos nodos de ROS: *Subscriber* y *Publisher*, encargados de recibir las estimaciones de la pose y publicar los comandos respectivamente. Del lado de la ESP32, hay dos nodos de uROS: *Subscriber* y *Publisher*, encargados de recibir los comandos, calcular las acciones de control de bajo nivel y publicar las magnitudes estimadas respectivamente. La Fig. 3b ilustra esta conexión.

IV. VALIDACIÓN

Para las pruebas experimentales de las adaptaciones realizadas en la Sección III-A, en esta sección se mostrarán los resultados obtenidos al aplicar la metodología de *continuidad de modelos* para el diseño de un control de seguimiento de *waypoints* (control de alto nivel) de un robot unicycle con dirección diferencial y dos ruedas. Este robot tiene una estructura sencilla y es representativo de una gran variedad de vehículos autónomos de tierra (UGVs, en inglés). Para ello, se propuso un experimento en el que el robot unicycle debe

¹Se empleó un control cinemático ya que muchos robots comerciales pueden seguir comandos de velocidad, aunque la misma arquitectura podría utilizarse para un control dinámico.

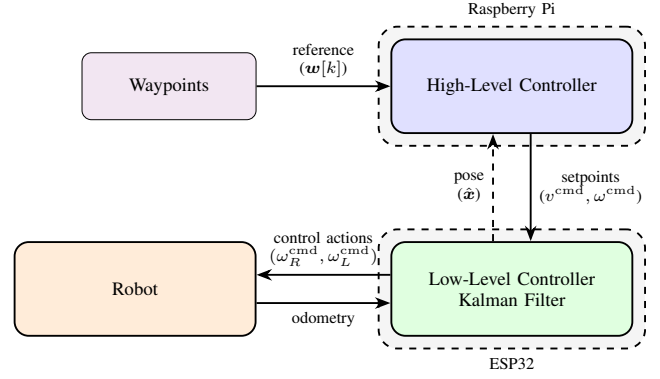


Fig. 1: Arquitectura del Control. Alto Nivel: Control Cinemático de Movimiento (Raspberry Pi 4). Bajo Nivel: Control de los Actuadores y Fusión de Datos de Sensores (ESP32).

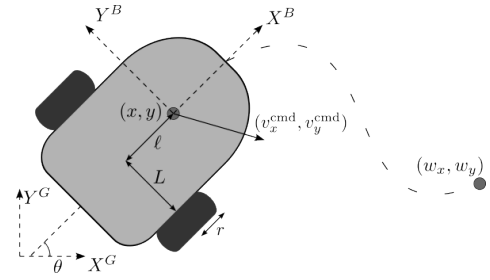


Fig. 2: Modelo de robot unicycle con dirección diferencial y dos ruedas.

visitar cuatro *waypoints* distribuidos en el espacio, formando un cuadrado de $1 \text{ m} \times 1 \text{ m}$.

A. Modelo Unicycle y Ley de Control

El robot unicycle se caracteriza por contar con tres grados de libertad, puede desplazarse hacia adelante o hacia atrás y rotar sobre su eje vertical, pero no posee capacidad de movimiento lateral (Fig. 2). La pose del unicycle se describe mediante el vector $x = (x, y, \theta)$, donde (x, y) es la posición de un punto ubicado sobre el eje X^B (terna local B) a una distancia $\ell > 0$ del punto medio del eje de las ruedas. Aquí, $\theta \in [0, 2\pi)$ es la orientación del eje X^B respecto al eje X^G de la terna global (G). Para este tipo de robot, la orientación θ no es controlable de forma independiente, sino que queda determinada por la velocidad comandada v^{cmd} . Por este motivo, la referencia ω^{cmd} no se utiliza.

1) *Control de Alto Nivel:* Se consideraron dos estrategias de control de seguimiento de waypoints. En ambos casos la acción de control corresponde a la velocidad comandada del punto x expresada en terna G: $v^{\text{cmd}} = (v_x^{\text{cmd}}, v_y^{\text{cmd}})$. Por un lado, un controlador PID que regula el error de posición $e[k] := (x[k], y[k]) - (w_x[k], w_y[k])$. Por otro lado, una ley de control basada en el gradiente descendente (GD) de la función de costo $\|e[k]\|^2/2$. Las acciones de control correspondientes

en cada caso resultan

$$\begin{aligned} (v_x^{\text{cmd}}, v_y^{\text{cmd}}) &= k_P e[k] + k_I \mathcal{I}_k(e) + k_D \mathcal{D}_k(e) \quad (\text{PID}), \\ (v_x^{\text{cmd}}, v_y^{\text{cmd}}) &= -k_G e[k] / \|e[k]\| \quad (\text{GD}), \end{aligned} \quad (1)$$

donde k_P, k_I, k_D, k_G son ganancias ajustadas empíricamente, y $\mathcal{I}_k, \mathcal{D}_k$ son los operadores de integración y diferenciación correspondientes a una discretización bilineal.

2) *Control de Bajo Nivel*: El nivel inferior de control (ver Fig. 1) se encarga del control de la velocidad de cada rueda. El mismo computa $\omega_L^{\text{cmd}}, \omega_R^{\text{cmd}}$ (2) a partir de las velocidades comandadas, es decir, de la salida del controlador de alto nivel (1). Luego, $\omega_L^{\text{cmd}}, \omega_R^{\text{cmd}}$ son reguladas mediante un controlador PID por rueda.

$$\begin{bmatrix} \omega_R^{\text{cmd}} \\ \omega_L^{\text{cmd}} \end{bmatrix} = \begin{bmatrix} 1/r & L/r \\ 1/r & -L/r \end{bmatrix} \begin{bmatrix} \cos(\theta) & \sin(\theta) \\ -\sin(\theta)/\ell & \cos(\theta)/\ell \end{bmatrix} \begin{bmatrix} v_x^{\text{cmd}} \\ v_y^{\text{cmd}} \end{bmatrix}. \quad (2)$$

Mediante esta transformación, asumiendo que el error del controlador de bajo nivel es cero, es decir, $(\omega_R, \omega_L) = (\omega_R^{\text{cmd}}, \omega_L^{\text{cmd}})$, se logra seguimiento exacto de la velocidad comandada, es decir, $(\dot{x}, \dot{y}) = (v_x^{\text{cmd}}, v_y^{\text{cmd}})$.

B. Plataforma de Hardware

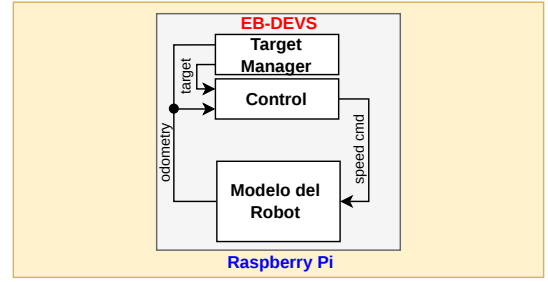
Se desarrolló una plataforma de hardware de un robot de tipo unicycle con dirección diferencial de bajo costo y fácilmente reproducible. La plataforma empleada cuenta con dos motores DG01D-E con encoder en cuadratura y una resolución de 500 pulsos por vuelta. Las mediciones de los encoders se utilizan para la estimación de la odometría del robot, y se combinan con datos de un magnetómetro HMC5883L mediante un Filtro de Kalman (ejecutado en la ESP32) para la estimación de la pose del robot $\hat{x}$. Tanto el control de bajo nivel de las ruedas como la estimación de la pose se hacen a una tasa de $T = 10$ ms.

En la ESP32, la tarea periódica de *spin* de uROS se encarga de manejar la suscripción a `/cmd_vel` (*history depth* igual 1) y el *timer* para la publicación en `/odom` (100 Hz). Además, la tarea de control se encarga de ejecutar periódicamente (100 Hz) el control de velocidad de las ruedas y el cálculo del filtro de Kalman. Para las mediciones de la orientación, se lee por I2C el registro correspondiente del HMC5883L, mientras que la lectura de la odometría se realiza por interrupción utilizando el módulo PCNT de la ESP32.

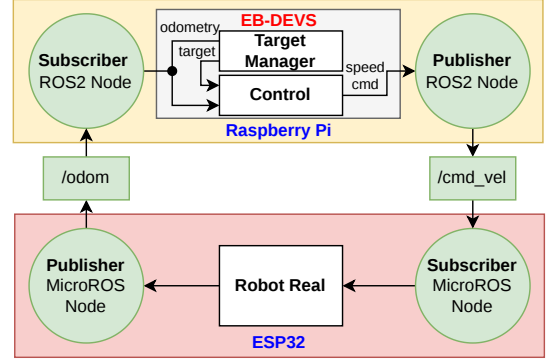
C. Verificación por Simulación del atómico Control

Esta es la primera etapa de la metodología de diseño basada en la *continuidad del modelo* del controlador propuesto.

El modelo completo utilizado en esta etapa corresponde a un único agente construido con la biblioteca de modelos EB-DEVS de la Sección II-B (Fig. 3a), con un modelo atómico *Control*, que implementa el control de seguimiento de waypoints, y un modelo acoplado *RobotDynamics*, que representa el comportamiento del unicycle definido por (2). Para este último, se utilizaron tres atómicos integradores *mQSSI*, para la posición y orientación del robot, con parámetros $dQ_{\min} = 0.001$ y $dQ_{\text{rel}} = 0.0$. El modelo *Control* se ejecuta periódicamente a 10 Hz. El atómico *Target_Manager* se encarga de proveer el próximo *waypoint* a visitar.



(a) Verificación por Simulación.



(b) Simulación Embebida.

Fig. 3: Etapas de la metodología de diseño del control de alto nivel por *Continuidad de Modelos*: (a) Verificación por simulación, (b) Validación experimental por simulación embebida en tiempo real sobre ROS2. El modelo *Control* es el mismo a lo largo de las distintas etapas.

La trayectoria resultante con el atómico *Control* implementando el Control PID de seguimiento de waypoints aplicado sobre el modelo del robot se presenta en la Fig. 4a. La Fig. 4b muestra la trayectoria seguida por el mismo modelo del robot pero con el atómico *Control* implementando el control basado en GD. Las estrellas indican los waypoints a visitar, mientras que los puntos verde y rojo indican las posiciones de partida y finalización respectivamente. En ambos casos se puede notar que el vehículo se dirige al waypoint correspondiente, y una vez que ingresa dentro del radio de aceptación (indicado en línea punteada) modifica su orientación para apuntar al próximo waypoint a visitar.

D. Validación por Simulación Embebida de Control

En esta segunda etapa de la metodología, se reemplazó el modelo del robot de la Fig. 3a por una interfaz de ROS2 con el robot real y la simulación se ejecuta en tiempo real. La Fig. 3b muestra la conexión entre el control de bajo nivel que se ejecuta en la ESP32 con el control de alto nivel que se ejecuta en el simulador EB-DEVS en la Raspberry Pi. Los modelos *Control* y *Target_Manager* son exactamente iguales a los de la etapa anterior, pero ahora reciben como argumento un objeto *ROS2Client* para comunicarse con ROS2 (basados en los atómicos *Publisher* y *Subscriber*

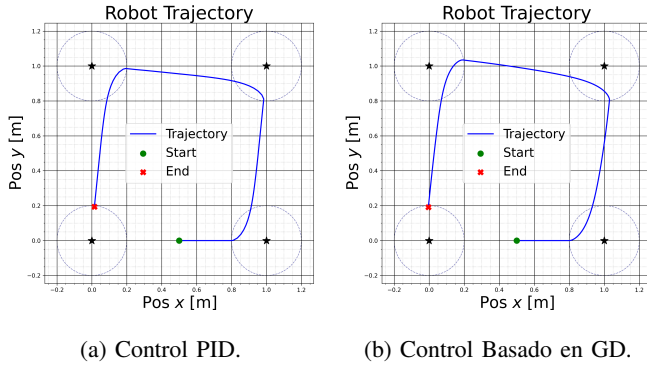


Fig. 4: Verificación por simulación del modelo atómico `Control` en EB-DEVS que implementa el control de seguimiento de waypoints ensayado sobre el Modelo del Robot. (a) Control PID. (b) Control basado en GD.

de la Sección III-A). Estos se comunican con los nodos `Publisher` y `Subscriber` de uROS en la ESP32.

La Fig. 5a muestra la trayectoria seguida por el robot real utilizando el modelo `Control` implementando el control PID de seguimiento de waypoints. La Fig. 5b muestra la trayectoria seguida por el robot real utilizando el modelo `Control` implementando el control de seguimiento de waypoints basado en GD. Se puede notar que el objetivo de acercarse a los waypoints se satisface, aunque la trayectoria real del robot muestra algunas discrepancias respecto a las obtenidas sobre el modelo del robot en la Fig. 4. Para cuantificar esta diferencia en el escenario embebido, se define como *trayectoria ideal* a la recta que une la posición del robot en el momento de conmutación de objetivo con el nuevo waypoint asignado. Respecto a esta referencia, el apartamiento máximo (Δp) medido es de 8.2 cm para el control PID, y de 7.8 cm para el control basado en GD. Es posible que este efecto se deba a la existencia de errores en la estimación del estado, a dinámicas de orden superior no contemplados, y/o a retardos en el tiempo asociados a la interfaz EB-DEVS/ROS2.

Para verificar la correcta ejecución del control dentro del simulador EB-DEVS embebido en tiempo real y los retardos en la interfaz EB-DEVS/ROS2, se utilizaron las herramientas de ROS2. El comando `ros2 topic hz` permite evaluar la frecuencia media de publicación en `/cmd_vel` y `/odom`. Con esto se verificó que la simulación embebida se ejecuta en tiempo real y la frecuencia promedio de publicación en `/cmd_vel` coincide con la frecuencia configurada en el atómico `Control` en EB-DEVS (10 Hz). También se observó una fuerte relación entre la frecuencia de publicación en `/cmd_vel` (EB-DEVS) y `/odom` (definida por un `timer`) y el período de las tareas de `spin` de uROS y de control en la ESP32. Para analizar si estas dos tareas están compitiendo por el uso de recursos se realizaron ensayos con el atómico `Control` ejecutándose a (f_{Control}): 100 Hz, 50 Hz, 20 Hz y 10 Hz, para la misma frecuencia promedio de publicación de la odometría ($f_{\text{odom}} = 100$ Hz). Se comprobó que sólo para el último caso f_{odom} era 100 Hz de forma sostenida. Si en

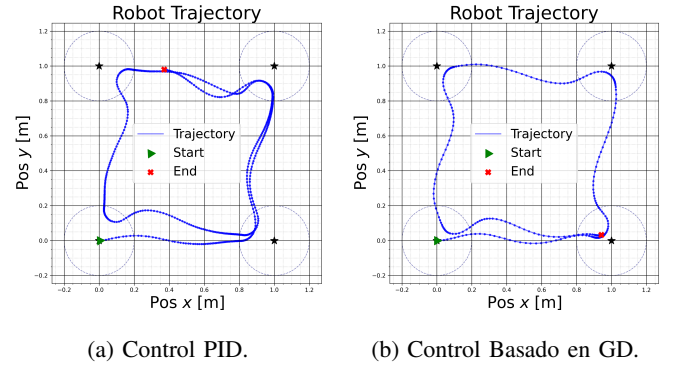


Fig. 5: Validación por *simulación embebida* del modelo atómico `Control` en EB-DEVS que implementa el control de seguimiento de waypoints ensayado sobre el robot real. (a) Control PID ($\Delta p = 8.2$ cm). (b) Control basado en GD ($\Delta p = 7.8$ cm).

cambio ambas frecuencias coinciden: $f_{\text{Control}} = f_{\text{odom}} = f$ con $f \in \{5 \text{ Hz}, 10 \text{ Hz}, 20 \text{ Hz}, 50 \text{ Hz}\}$, f_{odom} tiene el valor esperado de forma sostenida en todos los casos. En estas condiciones, el seguimiento de los *waypoints* presenta un apartamiento máximo Δp de 15.49 cm, 6.69 cm, 5.43 cm y 8.05 cm respectivamente.

V. CONCLUSIONES

En este trabajo se ensayaron las modificaciones realizadas en el simulador EB-DEVS para permitir que los modelos puedan comunicarse con ROS2 en tiempo de ejecución. Además se validó una plataforma de hardware de un robot unicycle con dirección diferencial. Con estos se aplicó de forma satisfactoria la estrategia de diseño de *continuidad del modelo del controlador* para el diseño de dos estrategias de control de alto nivel para el seguimiento de waypoints: Control PID y Control basado en GD. Esto permite resaltar las ventajas de esta metodología: (a) el mismo modelo de simulación del controlador (`Control`) se utilizó en todas las etapas de diseño, y (b) para ensayar distintas leyes de control alcanza con reemplazar el modelo atómico `Control` en EB-DEVS, mientras que las interfaces con el hardware y el control de bajo de nivel permanecen sin cambios.

Asimismo, la arquitectura de control propuesta resuelve y encapsula las tareas de configuración y manejo del hardware, el cálculo de las acciones de control para los actuadores y la adquisición de datos en la ESP32. De esta manera un diseñador solo debe preocuparse del diseño del control de alto nivel dentro del simulador EB-DEVS y en lenguaje Python, sin requerir conocimientos de hardware y software de bajo nivel.

Este artículo presenta avances de un trabajo en curso, cuyo objetivo final es utilizar los modelos de control para NMRS desarrollados y verificados en [9] para el control coordinado de una flota de UGVs de tipo unicycle.

REFERENCES

- [1] M. Moallemi and G. Wainer, “Modeling and simulation-driven development of embedded real-time systems,” *Simulation Modelling Practice and Theory*, vol. 38, pp. 115–131, 2013.
- [2] X. Hu and B. Zeigler, “Model continuity in the design of dynamic distributed real-time systems,” *IEEE Transactions on Systems, Man, and Cybernetics - Part A: Systems and Humans*, vol. 35, no. 6, pp. 867–878, 2005.
- [3] E. Pecker-Marcosig, J. I. Giribet, and R. Castro, “DEVS-over-ROS (DoveR): A framework for simulation-driven embedded control of robotic systems based on model continuity,” in *2018 Winter Simulation Conference (WSC)*. IEEE, 2018, pp. 1250–1261.
- [4] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, “Robot Operating System 2: Design, architecture, and uses in the wild,” *Science Robotics*, vol. 7, no. 66, p. eabm6074, 2022.
- [5] T. Balaguer, O. Simonin, I. G. Lassons, and I. Fanloni, “DANCERS: A Physics and Network Co-Simulator for Communicating Multi-Robot Systems,” in *2025 IEEE International Conference on Simulation, Modeling, and Programming for Autonomous Robots (SIMPAP)*, 2025, pp. 1–6.
- [6] S. Acharya, M. Bharatheesha, Y. Simmhan, and B. Amrutur, “A co-simulation framework for communication and control in autonomous multi-robot systems,” in *Proc. of the 2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. Oct. 01st-05th, Detroit, MI, USA, 11087-11094., 2023.
- [7] A. Testa, A. Camisa, and G. Notarstefano, “ChoiRbot: A ROS 2 Toolbox for Cooperative Robotics,” *IEEE Robotics and Automation Letters*, vol. 6, no. 2, pp. 2714–2720, 2021.
- [8] E. Soria, F. Schiano, and D. Floreano, “SwarmLab: a Matlab Drone Swarm Simulator,” in *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2020, pp. 8005–8011.
- [9] E. Pecker-Marcosig, J. F. Presenza, I. Mas, J. I. Alvarez-Hamelin, J. I. Giribet, and R. Castro, “Simulation of Decentralized Coordination Strategies for Networked Multi-Robot Systems with Emergent Behavior-DEVS,” in *2025 Winter Simulation Conference (WSC)*. IEEE, 2025, pp. 2740–2751.
- [10] D. Foguelman, P. Henning, A. Uhrmacher, and R. Castro, “EB-DEVS: A Formal Framework for Modeling and Simulation of Emergent Behavior in Dynamic Complex Systems,” *Journal of Computational Science*, vol. 53, p. 101387, 2021.
- [11] B. P. Zeigler, A. Muzy, and E. Kofman, *Theory of Modeling and Simulation: Discrete Event and Iterative System Computational Foundations*, 3rd ed. San Diego, CA, USA: Academic Press, 2018.
- [12] R. Castro, M. Bergonzi, E. Pecker-Marcosig, J. Fernández, and E. Kofman, “Discrete-event simulation of continuous-time systems: evolution and state of the art of quantized state system methods,” *SIMULATION*, vol. 100, no. 6, pp. 613–638, 2024. [Online]. Available: <https://doi.org/10.1177/00375497241230985>
- [13] K. Belsare, A. C. Rodriguez, P. G. Sánchez, J. Hierro, T. Kolcon, R. Lange, I. Lütkebohle, A. Malki, J. M. Losa, F. Melendez, M. M. Rodriguez, A. Nordmann, J. Staschulat, and J. von Mendel, *Micro-ROS*. Cham: Springer International Publishing, 2023, pp. 3–55.
- [14] R. Paredis and H. Vangheluwe, “Modelling and simulation-based evaluation of twinning architectures and their deployment,” in *Proceedings of the 14th International Conference on Simulation and Modeling Methodologies, Technologies and Applications SIMULTECH-Volume 1*, 170-182, 2024, Dijon, France, 2024, pp. 170–182.